



Tema 2: Introduccion al ML, DL y LLMs – Clase 5

Contenidos

- Recordemos el viaje
- ¿Cómo se entrena un GPT?
- ¿Cómo genera texto un GPT?
- Prompting básico
- Construyendo un mini-GPT
- Demo en vivo



Recordemos el viaje



Lo que ya sabéis

El recorrido del Tema 2 hasta aquí — cinco sesiones de viaje. Hoy llegamos a lo que prometimos.

S1 · ML supervisado y no supervisado

Entrenamiento, test, inferencia, métricas.

Característica							Label (conocida)
	Edad	Sexo	Peso	Test 1	Test 2	Test 3	¿Enfermo?
	35	V	74	No	Si	No	No
	19	M	56	Si	Si	No	Si
	52	M	70	No	No	Si	Si

Entrenamiento de un modelo de IA con la base de datos disponible.							
	60	V	89	Si	No	No	?
							Predicción
Inferencia o predicción utilizando el modelo entrenado.							

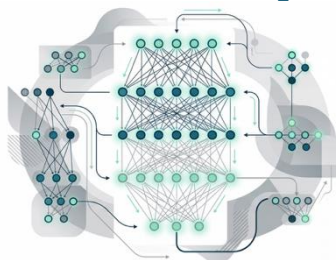
S3 · Infraestructura del DL

GPUs, CUDA, MPS, hiperparámetros.

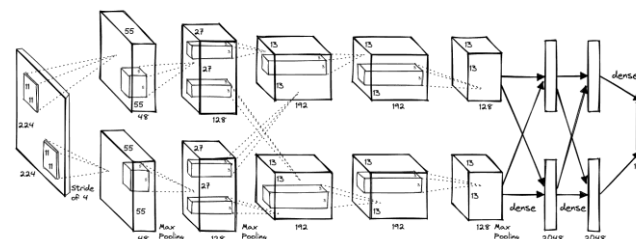


S5 · Hoy

El destino. GPTs: entender y construir.

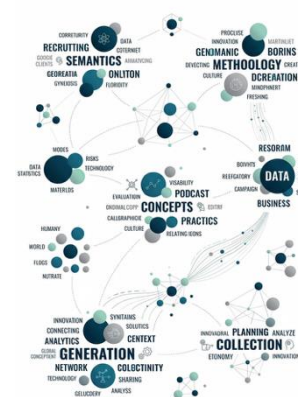


S2 · Deep Learning



Redes neuronales, overfitting, generalización.

S4 · NLP



Tokenización, embeddings, atención, Transformer, BERT vs GPT.



Hoy: entender y construir

Primera mitad · entender

- Cómo se entrena un GPT.
- Cómo genera texto.
- Cómo se le habla con prompting.
- ¿Y lo que no es texto?

Segunda mitad · construir

- Implementamos un mini-GPT desde cero en vuestro ordenador.
- Lo entrenamos sobre Don Quijote (~3 M parámetros, 5-8 min de entrenamiento).
- Lo usamos para generar texto al estilo Cervantes.



No solo veréis qué es un GPT. Lo tendréis funcionando en vuestro portátil.



¿Cómo se entrena un GPT?



La arquitectura

Recordatorio de la sesión anterior — a partir de aquí, la arquitectura ya no nos preocupa. Vamos a ver cómo se entrena.

Stack de bloques

N bloques idénticos apilados (típicamente 12-120).

Cada bloque

Masked Self-Attention + Feed Forward + normalizaciones + residuales.

Entrada

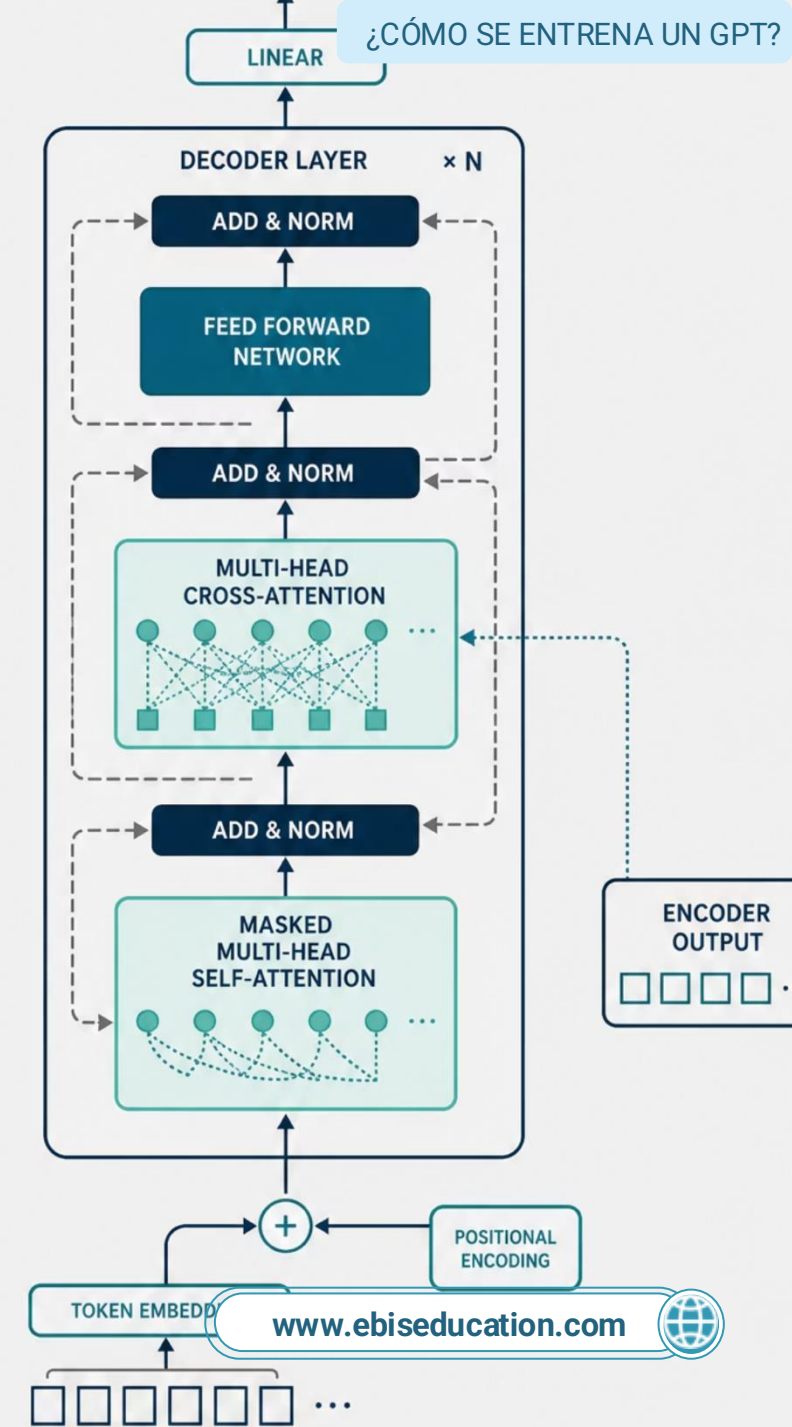
Embeddings de tokens + embeddings de posición.

Salida

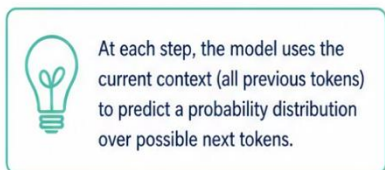
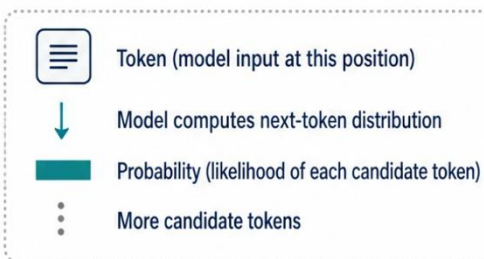
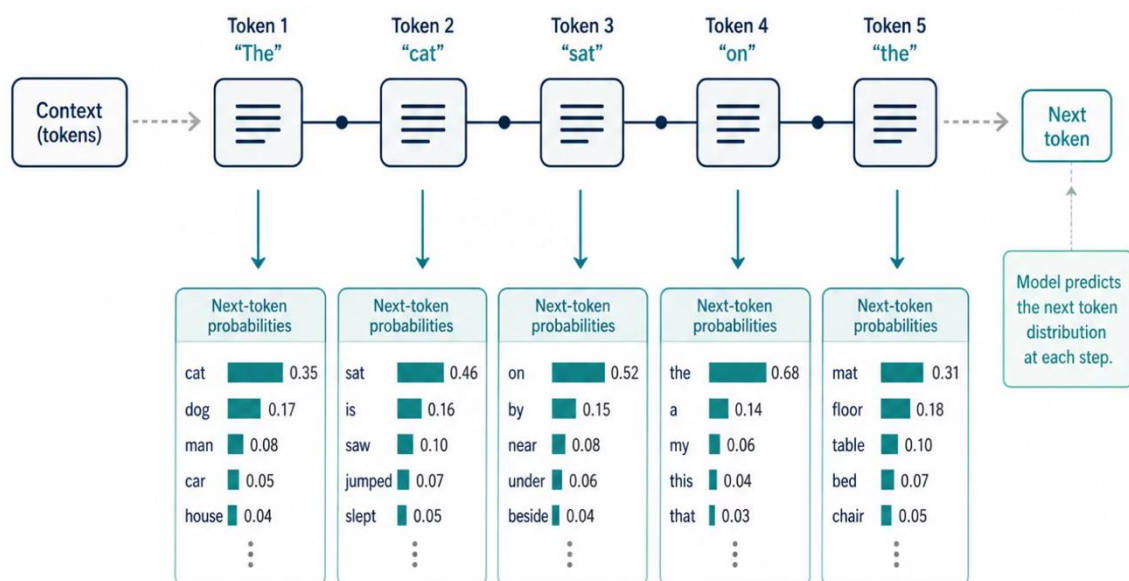
Distribución de probabilidad sobre el vocabulario para el siguiente token.

DECODER STACK

The decoder generates output sequences autoregressively by attending to the encoded context and previously generated tokens.



El objetivo: predecir el siguiente token



1 Una sola tarea

Dado un contexto de tokens, el modelo predice una distribución de probabilidad sobre el siguiente token.

2 Miles de millones de fragmentos

Durante el entrenamiento, le mostramos miles de millones de fragmentos de texto reales.

3 Ajuste de pesos

Para cada fragmento, le pedimos que adivine el siguiente token. Ajustamos los pesos según el error.



Self-supervised: no necesita etiquetas humanas. El "label" es el siguiente token del propio texto.



Cross-entropy – Cómo medimos el error

Predicción del modelo

Varias barras de distintas alturas – el modelo reparte probabilidad entre muchos tokens candidatos.

Realidad (one-hot)

Un 100% sobre el token correcto, 0% sobre los demás.

Loss = distancia

La función que el optimizador (Adam, SGD) intenta minimizar ajustando los pesos del modelo.

✓ Convencido y acierta

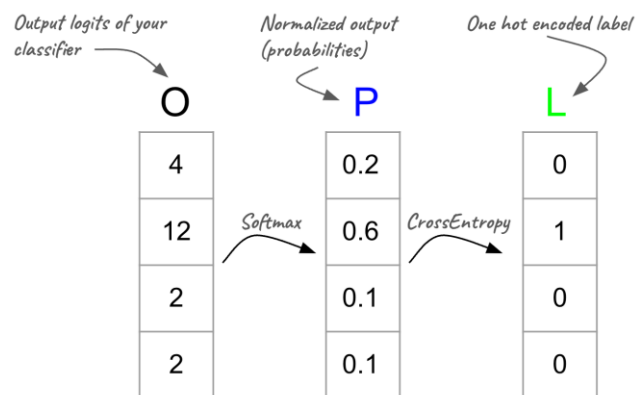
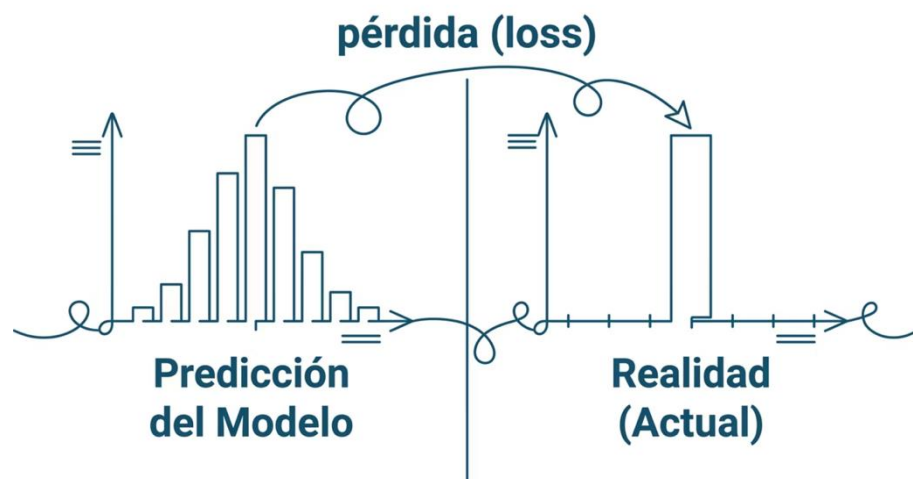
Loss muy baja.

✗ Convencido y falla

Loss muy alta.

🤔 Duda

Loss intermedia.



$$\text{CrossEntropy}(P, L) = - \sum_i L_i \log(P_i)$$



Datos, computación y scaling laws

300B

Tokens GPT-3

GPT-4 entrenado con más de 10T tokens.

175B

Parámetros GPT-3

GPT-1: 117M · GPT-4: ~1-2T (estimado).

~100M\$

Coste compute GPT-4

Entrenar GPT-4 costó ~100M USD en computación.

Scaling laws (Kaplan et al., 2020): Más datos + más parámetros + más cómputo → mejor modelo, de forma predecible. La predictibilidad es lo que ha permitido la inversión masiva en escala.



El gran descubrimiento de la década: **escalar funciona.**



¿Cómo genera un texto GPT?



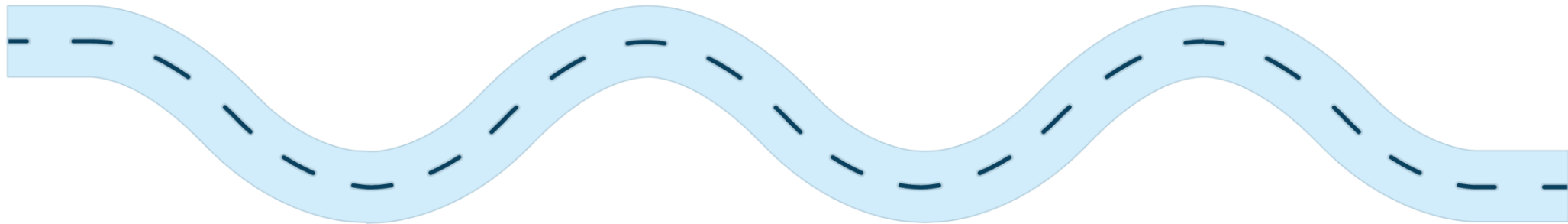
Generación autoregresiva – Token a token

Entrada

Recibe el prompt
inicial

Muestreo

Elige un token según la
probabilidad



Distribución

Calcula probabilidades
del siguiente token

Append

Añade el token y repite
el proceso

Un GPT no genera una frase entera de golpe. La genera token a token.

Por eso ChatGPT escribe palabra a palabra en pantalla. No es animación: es la generación real.

i Hasta que decide parar: token especial de fin, o max_tokens alcanzado.



Greedy vs sampling - ¿el mejor o un poco al azar?

Greedy decoding

- En cada paso, elegimos siempre el token más probable.
- Determinista: misma entrada → misma salida.
- Útil para: tareas factuales, resúmenes, código, traducción.
- Problema: tiende a respuestas repetitivas y aburridas.

Sampling

- En cada paso, elegimos un token muestreando según la distribución.
- Probabilístico: misma entrada → salidas distintas.
- Útil para: redacción creativa, brainstorming, conversación.
- Problema: puede irse por las ramas si no se controla.



La industria usa **sampling con controles**. Veamos cuáles.



Temperature – El dial de la creatividad



Temperature escala las probabilidades antes de muestrear:

$T = 0.0 \rightarrow$ Rígido

Equivalente a greedy (siempre el más probable).

$T = 0.7 \rightarrow$ Equilibrado

Ligero margen de creatividad. El valor por defecto recomendado.

$T = 1.0 \rightarrow$ Natural

Muestreo "natural" de la distribución original.

$T > 1.5 \rightarrow$ Caótico

Distribución muy aplanada. Texto incoherente.



Regla práctica: tareas factuales $\rightarrow$ temperature baja. Tareas creativas $\rightarrow$ 0.7-0.9.



Top-k y top-p - cortar la cola larga

Para evitar que el modelo elija tokens absurdos con probabilidad bajísima:

Top-k

- Solo considera los k tokens más probables.
- Resto: probabilidad 0.
- Típico: k = 50.

Top-p (nucleus sampling)

- Solo considera los tokens cuya probabilidad acumulada alcanza p.
- Adaptativo: en distribuciones "seguras" elige pocos, en distribuciones "dudosas" elige más.
- Típico: p = 0.9.



La práctica habitual: **temperature=0.7**, **top_p=0.9** y olvidarte de top-k.



Prompting básico



Anatomía de un prompt - roles

Una conversación con un LLM moderno se estructura en mensajes con tres roles:



system

Instrucción global que orienta el comportamiento del modelo.

"Eres un profesor de IA en EBIS, tono cercano, máximo 3 frases."



user

Lo que dice el usuario en cada turno de la conversación.



assistant

Lo que el modelo ya ha dicho antes (en conversaciones de varios turnos).

⚠ Los LLMs **no tienen memoria entre llamadas**. Hay que reenviar el contexto completo en cada llamada.

El **system prompt** es vuestro punto de control. Es donde está la magia.



Zero-shot vs Few-shot: Enseñar con ejemplos

Zero-shot

Pides la tarea sin ejemplos previos.

```
"Clasifica este review como positivo  
o negativo:  
«La pizza estaba fría»."
```

Few-shot

Das 2-5 ejemplos antes de la tarea real.

```
"Ejemplos:  
«Me encantó» → positivo.  
«Horrible servicio» → negativo.  
«La pizza estaba fría» →"
```

Cuándo usar zero-shot

Tareas comunes: resumen, traducción, sentimiento básico. Suele bastar.

Cuándo usar few-shot

Tareas raras, formatos específicos, o cuando zero-shot falla.



Few-shot es enseñar con ejemplos antes de pedir. Como con un becario nuevo.



Chain of thought – Piensa paso a paso

✗ Sin CoT

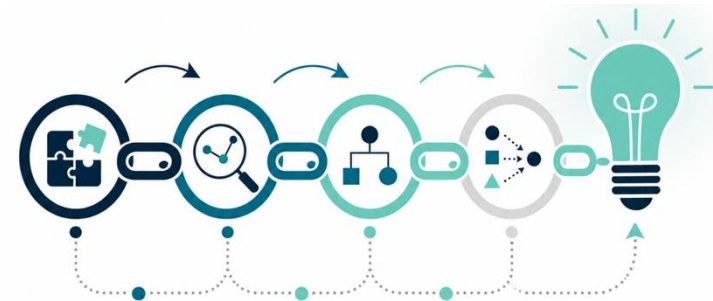
"Si Juan tiene 5 manzanas y compra 3 más, luego regala 4 a Ana, ¿cuántas le quedan?" → El modelo a veces falla.

✓ Con CoT

"... Piensa paso a paso." → "Juan empieza con 5. Compra 3 → tiene 8. Regala 4 → le quedan 4."



Funciona porque el modelo "ve" su propio razonamiento mientras lo escribe. Diferencia entre un 60% y un 90% de acierto en problemas multi-paso.



Reglas de oro del prompting



Sé específico

"Resume en 3 bullets" es mejor que "Resume".



Da contexto

El modelo no sabe quién eres ni para qué lo quieres.



Estructura la salida

Pide JSON, lista, tabla — formatos predecibles.



Usa ejemplos

Few-shot resuelve la mayoría de problemas raros.



Itera

El primer prompt nunca es el bueno. Prueba, ajusta, vuelve a probar.



Mide

Sin métricas, no sabes si tu prompt mejora o empeora.



El prompting es el 30% de la ingeniería con LLMs. Y se aprende practicando.



¿Prompting o fine-tuning?

Imaginad que queréis hacer un asistente experto en regulación bancaria española. ¿Por dónde empezáis?

A. Prompting

Un prompt bien escrito con ejemplos few-shot.

B. Fine-tuning

Fine-tuning del modelo con vuestros documentos.

C. RAG

Recuperar fragmentos relevantes y pasarlos como contexto.

D. Combinación

Combinación de varios enfoques.



¿Cuál elegís y por qué?



Construyendo un mini-GPT



¿Por qué hacemos esto?

"Cualquier perfil del Máster que entienda lo que entrena un mini-GPT entiende lo que entrena GPT-4.
La diferencia es de escala, no de naturaleza."

Lo que vais a hacer hoy:

→ Implementar la arquitectura GPT

En ~150 líneas de PyTorch.

→ Ver la evolución

Cómo el modelo pasa de generar caracteres aleatorios a generar texto con sintaxis cervantina.

→ Entrenar el modelo

Sobre el texto íntegro de Don Quijote.

→ En 5-8 minutos

En mi portátil.



No vais a salir capaces de entrenar GPT-4. Vais a salir sabiendo exactamente cómo funciona por dentro.



Las cifras de nuestro modelo

4

Bloques Transformer

Apilados en el modelo.

4

Heads de atención

Por bloque Transformer.

256

Dimensiones embedding

Espacio vectorial de representación.

128

Tokens de contexto

Máximo de contexto por inferencia.

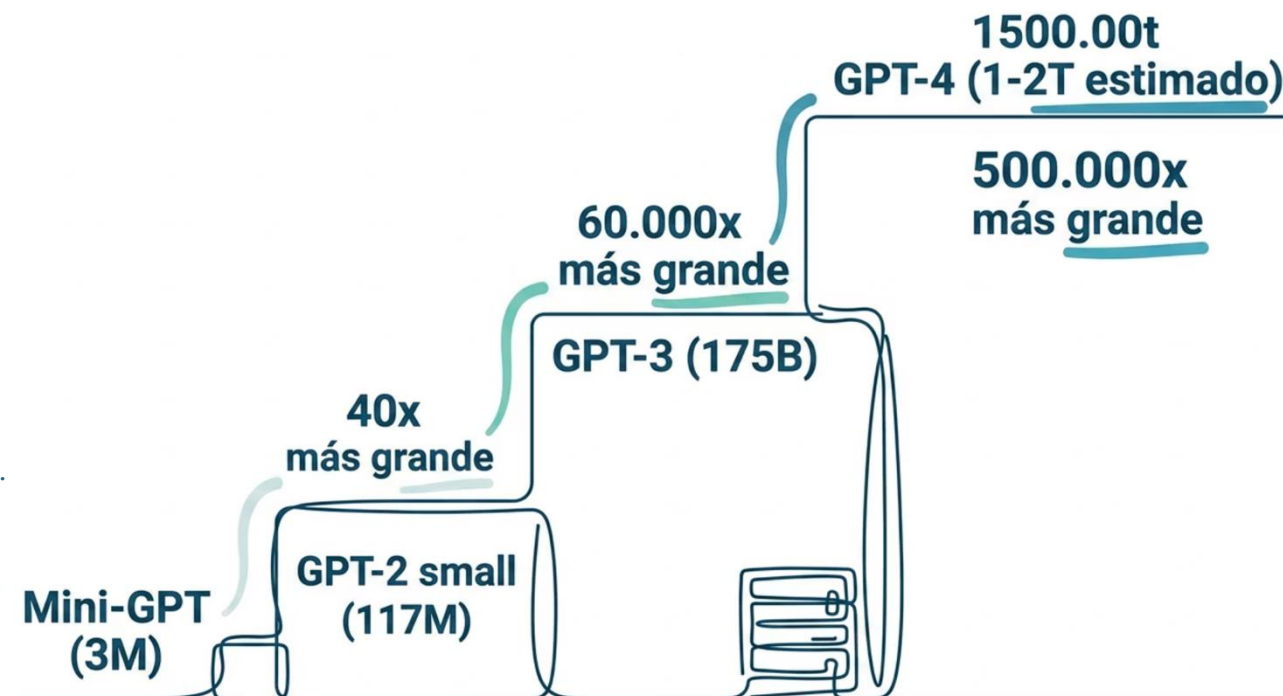
~90

Tokens vocabulario

Tokenización a nivel carácter.

3M

Parámetros totales



Las herramientas que vamos a usar



PyTorch

- Framework de Deep Learning estándar de la industria.
- API limpia, control total sobre el modelo.
- Lo veréis en todas las empresas de IA.



MPS · Metal Performance Shaders

- Backend de PyTorch para GPUs de Apple Silicon.
- 5-10× más rápido que CPU pura en M4.
- Activado con una sola línea:
`device = "mps".`

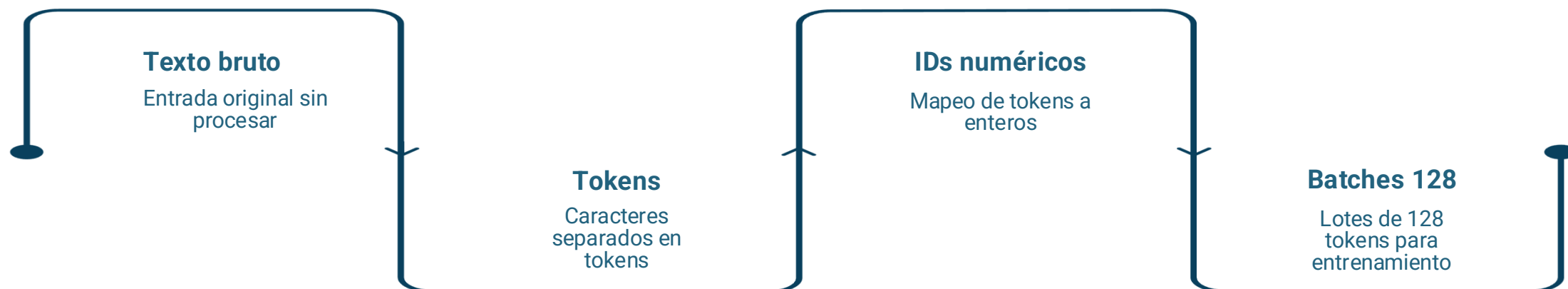


Don Quijote

- Texto íntegro en español, ~2 MB, dominio público.
- ~2.000.000 caracteres / ~90 caracteres únicos.
- Genera un modelo con sintaxis cervantina muy reconocible.



Pipeline de datos – texto a batches



01

Cargar el texto crudo

```
open("quijote.txt").read()
```

03

Codificar como tensor

Todo el texto como un tensor gigante de IDs.

02

Tokenizar a nivel carácter

Cada carácter único = un token con un ID.

04

Batchear

Extraer aleatoriamente secuencias de 128 tokens para entrenar.

Para cada batch, el modelo recibe el input (tokens 0..127 de una secuencia) y el target (tokens 1..128, los mismos desplazados una posición). Lo que estamos pidiendo: "para cada token, predice el siguiente".



El modelo – Estructura en código

```
class MiniGPT(nn.Module):
    def __init__(self):
        self.token_emb = nn.Embedding(vocab_size, n_embed)
        self.pos_emb = nn.Embedding(block_size, n_embed)
        self.blocks = nn.Sequential(*[Block() for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embed)
        self.head = nn.Linear(n_embed, vocab_size)
```

De arriba abajo, los componentes:

01

Embeddings de tokens

Tabla aprendida, ID → vector.

02

Embeddings de posición

Vector aprendido por posición en la secuencia.

03

N bloques Transformer

Cada uno: attention + feed-forward.

04

Layer norm final

Estabilización de la salida.

05

Head de salida

Proyecta a logits sobre el vocabulario.



~150 líneas en total. Lo veremos completo en el notebook.



El modelo – Estructura en código

El loop de entrenamiento, en pseudo-código:

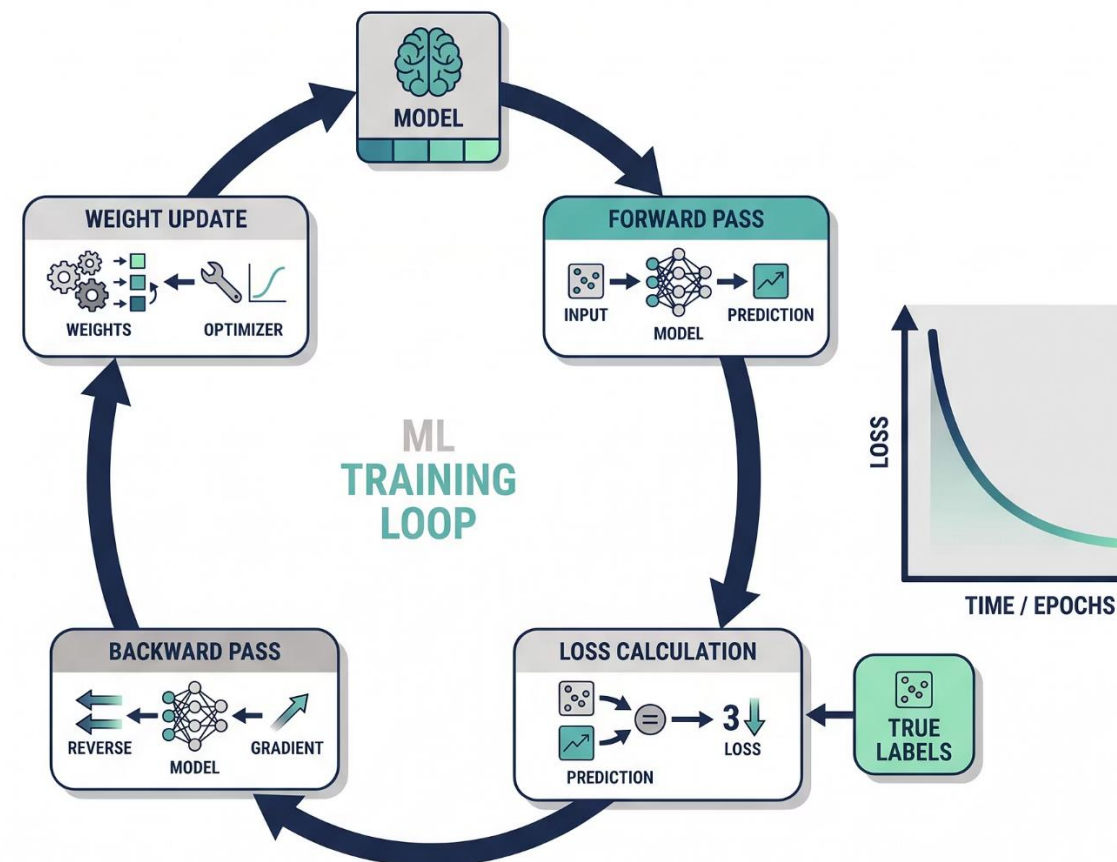
```
for iter in range(max_iters):
    x, y = get_batch()           # input + target
    logits = model(x)           # forward pass
    loss = cross_entropy(logits, y) # error
    loss.backward()              # calcula gradientes
    optimizer.step()             # actualiza pesos
```

Que se repite **~500 veces**.

Mientras tanto, vamos imprimiendo la loss cada 50 iteraciones. Veremos cómo baja de **~4.5** (caos total) a **~1.5** (texto cervantino reconocible) en pocos minutos.



Este es el mismo bucle que se usa para entrenar GPT-4. Mismo principio, distinta escala.



Inferencia – Generamos texto

```
def generate(prompt, max_new_tokens):  
    for _ in range(max_new_tokens):  
        logits = model(prompt[-block_size:]) # mira solo últimos 128  
        logits = logits[-1] / temperature    # toma el último, aplica T  
        probs = softmax(logits)  
        next_token = sample(probs, top_p=0.9) # nucleus sampling  
        prompt = torch.cat([prompt, next_token])  
    return prompt
```



Es exactamente lo que hace ChatGPT por dentro. Sin atajos.

Una vez entrenado el modelo, esta función nos da texto al estilo del corpus de entrenamiento. Todos los parámetros que hemos visto – temperature, top_p – están aquí, en acción.



Demo en vivo





EBIS Business
Techschool

www.ebiseducation.com

